SC4051 Distributed Systems

Course Project: Design and Implementation of A Distributed Banking System

PROJECT MEMBERS (SLOT 7: 10.00-10.15)

NO.	NAMES	MATRICULATION NUMBER	Percentage (%)
1	Koh Zhi Yong, Marcus	U2222814E	50
2	Toh Jian Hong	U2221561F	50

AY 2025/2026

SEMESTER 2

COMPUTER ENGINEERING / COMPUTER SCIENCE COURSE

1. Introduction 3
2. Message Format and Data Marshalling 3
2.1 Marshalling and Byte Ordering 3
2.1.1 Java Client Data Serialization: 3
2.1.2 C++ Server Data Serialization: 4
2.1.3 Floating-Point Representation: 4
2.1.3 Integers and Enum Types (Bank Account Number, Currency): 4
2.1.4 Variable-Length Strings: 4
2.1.5 Longs: 4
2.2 General Message Structure 5
2.2.1 Client Request Packets: 5
2.2.2 Server Reply Packets: 5
3. Operations 6
3.1 Open Account 6
3.2 Close Account 7
3.3 Deposit 7
3.4 Withdraw 8
3.5 Monitor 8
3.6 Additional Idempotent Operation: Check Balance 9
3.7 Additional Non-idempotent Operation: Transfer Funds 9
4. Invocation Semantics Design 10
4.1 At-Least-Once (ALO) Semantics 11
4.2 At-Most-Once (AMO) Semantics 11
5. Failure Simulation and Experiments 12
5.1 Experiment Setup & steps to reproduce 13
Experiment Results: ALO vs AMO Semantics 13
6. Conclusion 14
Appendix: 15
ALO 15
AMO 19

1. Introduction

This report outlines the design and implementation of a Distributed Banking System utilizing a client-server architecture. The Server is implemented in C++, while the Client is implemented in Java. All inter-process communication in our project is handled via UDP sockets, using careful byte-level data marshalling to ensure cross platform compatibility.

The system supports the five required core banking operations stated in the lab manual: opening an account, closing an account, deposit, withdrawals, interval based callback monitoring service, as well as an additional idempotent and non-idempotent operation. Finally, to address the inherent unreliability of UDP, the system implements and evaluates both At-Least-Once and At-Most-Once invocation semantics under simulated network failure conditions detailed in our experiments.

2. Message Format and Data Marshalling

Since UDP is a connectionless, message-oriented protocol, all communication between the Java client and the C++ server must be explicitly packaged into discrete, self contained packets (datagrams) before transmission over the network. Every piece of information must be serialized (marshalled) into a standardized bye array by the sender, and subsequently deserialized (unmarshalled) by the receiver.

2.1 Marshalling and Byte Ordering

Mixing C++ and Java introduced a classic networking hurdle: endianness. Java operates on the Java Virtual Machine (JVM), which strictly defines all multi-byte primitive data types such as `int` and `float` in **Big-Endian** format. Conversely, C++ compiles directly to machine code. When the C++ server is executed, it inherently uses **Little-Endian** format. If the Java client were to send a 32-bit integer as a raw byte of sequence, the C++ server would interpret those raw bytes in reverse order, resulting in catastrophic logic failures, such as corrupted account numbers etc.

To resolve this fundamental incompatibility, we enforce a strict convention: all binary data transmitted over the UDP socket must be formatted in Network Byte Order, which is universally defined as Big-Endian.

2.1.1 Java Client Data Serialization:

The java implementation elegantly handles translation by wrapping byte arrays in the ByteBuffer class and explicitly by definition the byte order via `buffer.order(ByteOrder.BIG_ENDIAN)`.

This ensures that all chained `putInt()` and putFloat() operations automatically place the most significant byte first in the payload.

2.1.2 C++ Server Data Serialization:

The C++ implementation manually handles byte swapping. For 32-bit signed and unsigned integers the server utilizes the standard socket functions `htonl()` (host-to-network-long) before transmission, and `ntohl()` (network-to-host-long) upon receipt to swap bytes if the host system is Little-Endian.

2.1.3 Floating-Point Representation:

Floating point numbers are trickier since standard socket libraries do not provide `htonl()` equivalents for floats. To reliably marshal account balances, the C++ server implements a custom workaround that first copies the 32-bit IEEE 754 float directly into an unsigned 32-bit integer (`uint32_t`) using `memcpy`. It then safely applies `htonl()` to swap the bytes without altering the underlying representation, finally appending it to the transmission buffer.

2.1.3 Integers and Enum Types (Bank Account Number, Currency):

Integers are cast to their unsigned 32-bit form before being marshalled in C++. In Java, the bytes are placed directly into an initialised byte array (following Big Endian formatting), via the ByteBuffer class with `putInt()`. An unsigned integer type does not exist for Java. The workaround is to read the bytes as an integer before casting as a long before masking only the first 32 bytes. Enumerated types, or Enums, define a set of named Integer constants. Enum types are marshalled as their Integer counterparts then converted back and stored in an Enum type variable.

2.1.4 Variable-Length Strings:

To marshall strings, the string length followed by the UTF_8 string characters are written into a prepared buffer before transmission. In the case of C++, string length returned by the length operator is `size_t`, which varies from system to system and thus for consistency, must be cast to an unsigned integer of fixed length. `uint32_t` was chosen in this case, since the server does not need to send strings exceeding lengths of $2^{32} - 1$. In the case of Java, `putInt()` is used to marshall the string length. The character array is then copied in, and since a character is only a byte-wide, it does not need to be converted based on endianness. String lengths will be converted using htonl() in C++ before being written.

2.1.5 Longs:

Although Java's ByteBuffer library provides `putLong()` as an operation to easily marshall Longs in Big Endian format, unmarshalling at the server is more complicated since ntohl() only accepts 32 bits of input. The workaround is to read in the first 32-bits, ntohl(), then cast and left bit shift by 32 bits, before reading in the next 32 bits and doing a bit-wise OR operation.

2.2 General Message Structure

To simplify the unmarshalling process, ensure efficient processing, and minimize the risk of parsing errors, every UDP payload exchanged between the Java client and the C++ server follows a strictly defined, standardized envelope structure. For the reply and response message structures, we assume that both the server and client programs have common knowledge of the order and types of the data items in a message, like the CORBA CDR covered in the lectures. Data is tightly packed into containers/buffers without padding before transmission. The unmarshalling logic then copies data bytes at a time into initialised variables for later business logic use, as covered section 2.1.

2.2.1 Client Request Packets:

Every request dispatched by the client begins with an identical 8-byte header, providing the server with the necessary context to route and process the variable payload that follows. The header consists of two primary integer fields (OpCode, Request ID):

OpCode (4 bytes): This integer serves as a routing identifier that directly maps to the user's requested service. The mapped operations range from 1 (Open Account) to 7 (Transfer Funds). On receiving a request, the server strips and parses the first four bytes as the opcode to determine which service was requested.

Request ID (4 bytes): The java client program generates a unique, monotonically increasing sequence number for every distinct request it transmits. This 32-bit integer is the backbone of the system's fault tolerance strategy; it allows the server to track exactly which requests have already been fulfilled, enabling the server to resend cached responses under an At-Most-Once (AMO) invocation semantic, in particular, when the server receives a retransmitted request from the client.

The remaining bytes of the request packet are highly variable, dynamically constructed to house the specific arguments required for the declared OpCode (e.g., strings containing passwords or IEEE-754 floats representing deposit amounts).

2.2.2 Server Reply Packets:

The Server replies follow a similar template. They always start by echoing back the original request's OpCode(4 bytes). Because UDP doesn't guarantee packet order, this is crucial for the asynchronous client to verify exactly what context the incoming data belongs to.

Immediately following the OpCode, the server transmits a length-prefixed Status String. Successful operations transmit "SUCCESS", or "ACK" for callbacks. Failed operations transmit detailed error identifiers (e.g., "ERROR_ACCOUNT_NOT_FOUND", "ERROR_INSUFFICIENT_BALANCE"). This effectively acts as a secondary branch for the client GUI; if successful, it parses the mutated data (like updated balances), if it fails, it reads the error log to safely update the user interface.

3. Operations

Building upon the 8-byte header defined in Section 2.2, the system implements seven distinct services. Due to UDP's lack of native object serialization (unlike Java RMI or RPC), each operation utilizes a tailored byte layout for request and reply payloads. The two functions that we have chosen to fulfill the extra idempotent and non-idempotent operations are Check Balance and Transfer Funds respectively.

3.1 Open Account

This primary service handles the instantiation of a new account entirely on the server-side. The client constructs the packet by evaluating user input, marshaling variable-length string fields for identity (name and password), standardizing the starting balance as a 32-bit floating point, and mapping currency effectively into a uint32_t. A user is able to create two separate accounts with the same name, password and balance amount.

The client program also handles input checking for the currency and balance fields:

- The user is allowed to input one of three currency options, USD, JPY or SGD (case-insensitive).
- The user is allowed to input a positive balance. A negative balance, or any input other than a recognisable float will cause the client program to exit and prompt the user again for his/her choice of service.

int opcode (1 for Opening Account, **4**)	int requestId (automatically increments for each request, **4**)	int name_length (length of the name string, **4**)
String name (**n**)	int password_length (**4**)	String password (**p**)
int currency_type (from enum Currency, where 1 for USD, 2 for JPY, 3 for SGD) (**4**)	float balance (**4**)	

Figure 1: Request message format for account creation, byte size in **bold** (total size: *24+n+p* bytes)

Upon successful creation, the server dynamically generates and returns a 32-bit integer identifier mapped as the Account Number. The account number increments on every open account request that it receives.

int opcode (**4**)	int response_type_length (**4**)	String response_type ("SUCCESS", **7**)
uint32_t created_account_number (**4**)		

Figure 2: Response message format (total size: *19* bytes). This response message is sent to clients monitoring for updates. No error messages are sent, so there is only one response_type, "SUCCESS".

3.2 Close Account

Once an account exists, this service facilitates the deletion of a user's remote records. The provided name and password must match against the server's in-memory copy before authorizing the account accountDatabase.erase() operation.

int opcode (2 for closing accounts, **4**)	int requestId (**4**)	Int name_length (**4**)
String name (**n**)	int account_number (**4**)	int password_length (**4**)
String password (**p**)		

Figure 3: Request message sent to the server (Total size: *20+n+p* bytes)

If the account number provided by the client is not recognised by the server/already deleted, the operation will fail, and the server will send a response containing an error message. If the account can be found otherwise, the server will send a response notifying the client who initiated the request and other monitoring clients of the successful operation (monitoring clients will not receive an update if the operation was unsuccessful).

int opcode (2 for Closing account) (**4**)	Int response_type_length (**4**)	String response_type ("SUCCESS") (**r**)
int success_msg_length (**4**)	String success_msg_length (**s**)	

Figure 4: Response message sent on a successful `close-account` operation. (Total size: *16+r+s* bytes).

The response message sent on an unsuccessful `close-account` operation is structurally the same, instead, the response type will be "ERROR_ACCOUNT_NOT_FOUND" with an appended error message.

3.3 Deposit

This service executes a deposit on a specified account. The client marshals the exact sequence required for authentication alongside a target amount the user wishes to deposit. If the currency types do not match between the account balance and the amount deposited, we convert between currency types using hardcoded exchange rates before adding.

	USD	JPY	SGD
USD	1	149.50	1.35
JPY	0.0067	1	0.009
SGD	0.74	111.0	1

Figure 5: Exchange rates

Assuming validation clears (deposited amount inputted at client is positive), the server responds by sending the updated account balance, together with the account number, amount, currency

type, new balance and its currency type (to make it clear to other monitoring clients which account has been successfully updated).

int opcode (3 for Depositing amount, 4 for withdrawing, **4**)	int requestId (**4**)	int account_number (**4**)
int name_length (**4**)	String name (**n**)	int password_length (**4**)
String password (**p**)	float amount (to be withdrawn/deposited) (**4)**	int currency_type (converted from enum Currency) (**4)**

Figure 6: Request message identical for both deposit and withdrawal services. (Total size: *28+n+p* bytes)

int opcode (**4**)	int request_type_length (**4**)	String request_type (**r**)
uint32_t account_number (**4**)	float amount (**4**)	uint32_t amount_currency_type (converted from enum Currency) (**4**)
float updated_balance (**4**)	uint32_t account_currency_type (**4**)	

Figure 7: Response message identical on successful deposit/withdrawal operations. Response messages sent due to erroneous operations mirror Figure 4 (with “ERROR_ACCOUNT_NOT_FOUND” for both deposit and withdraw) (Total size: *28+r* bytes)

3.4 Withdraw

Functioning as a direct counterpart to Deposit, the withdrawal service requests a state mutation but introduces additional server-side enforcement. Before processing the serialized deduction amount against the cache, the server checks whether there is sufficient account balance to make the withdrawal, by first converting the amount to the account currency type. If there is insufficient account balance, the server will respond with an error message matching the format in Figure 4, with a reply type: “ERROR_INSUFFICIENT_BALANCE”.

3.5 Monitor

For the monitor service, the client will send the monitor interval to the server which indicates the duration in which the server program will send updates to the client on any successful operation. The user is only allowed to input granularity up to the number of seconds. Each operation holds a reference to a list of client IP address/port records which it uses to send the ‘update’ message. Once the monitor interval expires, the client record is removed from the list.

To implement this, the server program will create a background thread whose responsibility is to delete client records when the corresponding monitor interval has expired. The client record itself is actually a struct, containing, sockaddr_in (containing client IP address and port), as well as an expiry. Before adding the record to the list, the expiry is calculated based on a monotonic clock (C++’s steady_clock) plus the monitor duration (received from client program as a long, in nanoseconds; unmarshalling covered in 2.1.5). The ‘list’ is implemented by a custom class, IterablePriorityQueue, which inherits from C++’s standard priority queue. This is so that the list

functions as-is (iterable), while benefiting from min-heap functionality. The first element of the list is thus always the client record whose expiry is the earliest.

The background thread waits until a new record has been added or a client record has expired. In the event of expiry, the thread reacquires the lock to pop the record off the queue. Otherwise, if the newly added record expires earlier than the record it is currently tracking, it switches to sleep until the earlier record expires. Correspondingly, the main thread must acquire the lock in order to insert a record into the queue. It releases the lock before notifying the background thread of the addition.

The server program will send an acknowledgement to the client. Upon receiving this, the client program will set the SO_TIMEOUT to the monitor interval and awaits for a response/update. The SO_TIMEOUT blocks the client program for the specified interval until it receives an update. On every update, the SO_TIMEOUT is set to the remainder of the interval until receive() automatically fails and passes an empty byte array to the client program. Monotonic clocks (System.nanoTime()) are also used to implement this.

Please note that for simplicity, we assume that the client and server are on the same network segment (as in the lab), and therefore transmission delays or RTT/2, which may otherwise cause the client interval to start later than the server interval, can be assumed to be orders of magnitude smaller than 1s or than what is humanly noticeable (also assuming no congestion, etc). For simplicity, we also assume that the server does not need to retransmit updates if the client ends up not receiving them (which would otherwise have to involve the implementation of sliding windows and acknowledgements).

3.6 Additional Idempotent Operation: Check Balance

An operation is defined as idempotent if executing it multiple times yields the exact same server state as executing it only once. Checking a balance is a perfect example of a read-only query. Whether the client requests the balance once or multiple times, the server's account database is never mutated, and the returned balance will never be incorrectly altered by the duplication.

The user provides his name, password, and account number. The client encapsulates these fields in a request (together with the opcode and requestId as covered) and sends it over to the server, which responds with the account number and balance used for presentation to the user in the terminal-based interface. An error message is returned instead if the requested account cannot be found.

3.7 Additional Non-idempotent Operation: Transfer Funds

Conversely an operation is non-idempotent if processing it multiple times causes the system state to change consecutively. We will elaborate on how to deal with this in Chapter 4.

Additionally, our design supports cross-currency transfer logic as well, utilizing an exchange rate (Figure 5) matrix on our server. The user specifies the source account number of the transfer (assumed to belong to the user), followed by his password, then the destination account and the amount to transfer. The server first verifies both accounts exist, computes the fractional conversion rate, subtracts from the source, and adds the converted total to the destination. Transfers between the same account are not allowed. Because the reply payload must securely inform the client of the exchange rate math, it utilized the longest byte array design in the system.

int opcode (7 for transfer, **4**)	int reply_type_length (**4**)	String reply_type (**r**)
uint32_t source_account_number (**4**)	uint32_t destination_account_number (**4**)	float source_amount (converted to source account's currency type) (**4**)
float destination_amount (converted to destination account's currency type) (**4**)	float new_source_account_balance (after transfer) (**4**)	float new_destination_account_balance (after transfer) (**4**)
float exchange_rate (**4**)	int source_account_currency_type (**4**)	int destination_account_currency_type (**4**)

Figure 8: Response message format for transferring amounts between accounts. (Total size: *44+r* bytes)

Error messages will be sent for various cases. The following lists the various error types:

- ERROR_ACCOUNT_NOT_FOUND: if the source account cannot be found with the account number provided, or an incorrect password was entered,
- ERROR_DEST_NOT_FOUND: if the destination account corresponding to the account number provided cannot be found.
- ERROR_SAME_ACCOUNT: if the source and destination accounts are one and the same,
- ERROR_INSUFFICIENT_BALANCE: if the source account does not have enough funds to make the transfer.

4. Invocation Semantics Design

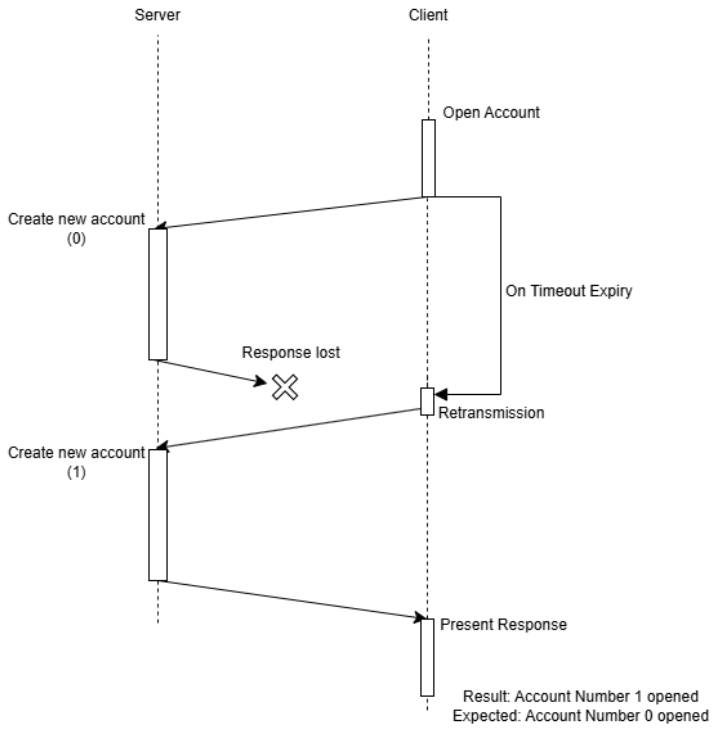
As UDP is connectionless, requests and replies may be dropped by the network. To guarantee that a client's request is eventually served, the Java Client implements a Timeout and Retry Mechanism. The client program sets the SO_TIMEOUT and expects to receive a response within the timeout interval. We have set this interval to 2 seconds. A Socket Timeout exception will be thrown if a response is not received within that time, in which case the client will retransmit the same request. We have set the client to retry 5 times before exiting and booting the user back to the main menu.

However, this retry mechanism will cause issues if the server receives duplicate requests due to retransmissions caused by response loss. How the server reacts, whether it dangerously executes the transfer a second's time or safely ignores it, depends entirely on the invocation semantics model currently active over the socket.

4.1 At-Least-Once (ALO) Semantics

When the C++ server is launched in ALO mode (server.exe alo), the system acts completely stateless regarding past requests. Every time a datagram arrives at the UDP socket, the server unpacks it, routes it by the OpCode, and executes the corresponding service function.

ALO guarantees that an application operation is executed at least one time. However, ALO is only safe for idempotent operations (like “Check balance”) where accessing the data multiple times does not change the persistent state. For inherently non-idempotent operations (like “Transfer”, “Withdraw”, “Deposit”), ALO is catastrophic, if the server receives a duplicate transfer packet due to a client retry, the ALO server simply process the payload again, improperly deducting identical amounts from the source account sequentially for example.

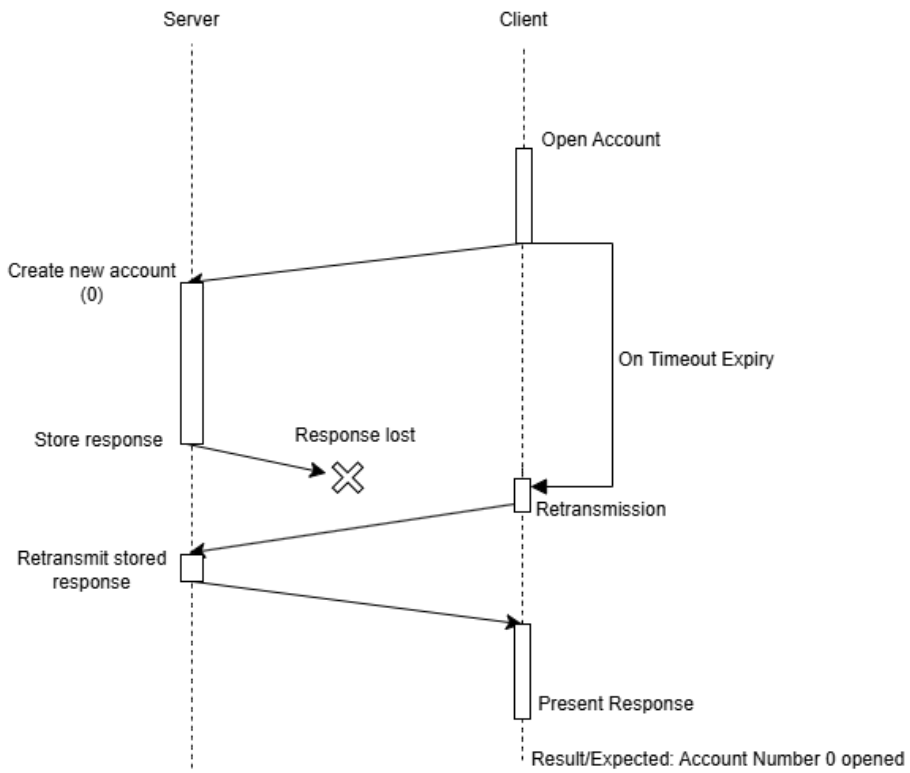


4.2 At-Most-Once (AMO) Semantics

When the system is launched in AMO mode (server.exe amo), the server trades memory for strict fault tolerance. AMO guarantees that an operation is executed exactly once, completely protecting non-idempotent operations from duplicate execution. To achieve this, a caching strategy and filtering logic are required.

To implement this, the C++ server maintains a requestHistory cache utilizing a Hash Map. Whenever a request arrives, the server concatenates the client’s socket address (IP and Port) with the 32-bit Request ID contained in the payload header to establish a unique transaction key.

Before invoking any operations across the database, the server queries the requestHistory map. In the event of a Cache Miss (a new request), the server processes the requested service normally. Crucially, before returning the result, it logs the final resulting Reply Byte Array into the hash map corresponding to the unique key. In the event of a Cache Hit (a duplicate request), the server recognizes the transaction key and completely bypasses executing the service function entirely. Instead, it pulls the specific parsed Reply Payload directly from the requestHistory cache and retransmits it down the UDP socket. This successfully answers the client's timeout retry while structurally shielding the non-idempotent variables. Please note that for ALO, if all retries fail, the operation is still executed at most once; the client just fails to receive the acknowledgement.



5. Failure Simulation and Experiments

To evaluate how our Distributed Banking System handles network unreliability, we designed a Failure Simulation feature. The server can be artificially configured to drop incoming packets (--drop_request) and (--drop_reply) using pseudo-random probability.

The most critical test compares At-Least-Once (ALO) and At-Most-Once (AMO) semantics when manipulating a non-idempotent operation under heavy reply drops. When an outgoing reply is dropped, the server has already mutated the state, but the client is oblivious, eventually timing out and automatically retrying the request.

5.1 Experiment Setup & steps to reproduce.

Setup: drop 50% of the outgoing replies, simulating a highly congested network where the server processes work, but clients frequently disconnect. Below is a detailed sample command chain demonstrating how we executed this specific scenario:

Command Chain 1: Testing ALO Semantics

1. **Server Execution:** `.\server.exe alo --drop_request=0.0 --drop_reply=0.5`
2. **Client Execution:** `java MainApp`
3. **Input Scenario:** Open Account (initial balance: $100.00), followed by Withdrawal request ($10.00).
4. **Observation:** The client times out due to simulated reply drops and initiates immediate retransmissions.
5. **Final Result:** The stateless server processes every duplicate request. This results in multiple withdrawals for a single intent, leaving an incorrect final balance of $80.00.

Command Chain 2: Testing AMO Semantics

1. **Server Execution:** `.\server.exe amo --drop_request=0.0 --drop_reply=0.5`
2. **Client Execution:** `java MainApp`
3. **Input Scenario:** Open Account (initial balance: $100.00), followed by Withdrawal request ($10.00).
4. **Observation:** The client times out due to simulated reply drops and predictably initiates immediate retransmissions. However, the server actively logs "Duplicate request detected" and refuses to re-execute the withdrawal operation. It intercepts the duplicate RequestID and safely re-transmits the cached reply.
5. **Final Result:** Retries are gracefully filtered out, leaving a correct final balance of $90.00.

Experiment Results: ALO vs AMO Semantics

The following table contrasts the findings for every banking operation built into the system during identical dropped-reply configurations:

Operation	Type	Result under At-Least-Once (ALO)	Result under At-Most-Once (AMO)
1. Open Account	Non-Idempotent	Fails (Duplicate Accounts): Retries cause the server to generate multiple new Account IDs for the same user, rather than returning the originally generated one.	Succeeds: Returns the exact same Account ID from the cached reply. No duplicate accounts created.
2. Close Account	Idempotent*	Fails (False Positive Error): The first request closes the account. The retry attempts to close it again, returning an ERROR_ACCOUNT_NOT_FOUND to the user instead of a SUCCESS message.	Succeeds: The server spots the duplicate request and returns the original SUCCESS closure message.

3. Deposit	Non-Idempotent	Fails (Double Deposit): The server processes the deposit multiple times. Re-transmissions incorrectly inflate the user's balance.	Succeeds: Duplicate requests are filtered. The server balance is only increased once.
4. Withdraw	Non-Idempotent	Fails (Double Withdraw): The server processes the withdrawal multiple times, potentially draining the user's account incorrectly.	Succeeds: Duplicate requests are filtered. Funds are strictly deducted only once.
5. Monitor Account	Non-Idempotent	Fails (Duplicate Registrations): The client's IP and port are registered multiple times in the callback list, causing them to receive duplicate event broadcasts.	Succeeds: The server recognizes the duplicate monitor request and ignores it, sending the cached ACK.
6. Check Balance	Idempotent	Succeeds: Retrying simply looks up the balance again. No state is mutated, so no harm is done.	Succeeds: The duplicate is caught and the cached balance is returned securely.
7. Transfer Funds	Non-Idempotent	Fails (Double Transfer): Funds are drained from the source and duplicated in the destination account repeatedly with every retry.	Succeeds: Duplicate requests are filtered. The transaction occurs exactly once.

6. Conclusion

In this project, we successfully designed and implemented a cross-platform Distributed Banking System utilizing low-level UDP socket programming, We gained a concrete understanding of the fundamental mechanics of distributed networking, including manual byte-level marshalling, edinaness resolution across C++ and Java architectures, and the lifecycle of client-server message passing.

The system securely supports critical banking operations, ranging from idempotent reads like checking balances to complex non-idempotent state mutations like cross-currency fund transfers. To evaluate the robustness of this system, we engineered a pseudo-random network failure simulation. Through explicit testing, we empirically proved the danger of At-Least-Once (ALO) invocation semantics in unreliable networks, where packet loss inevitably cascades into catastrophic data duplication for non-idempotent operations.

Conversely, we demonstrated that At-Most-Once (AMO) semantics gracefully abstract away the unreliability of the lower network layers. By caching server responses and aggressively filtering duplicate RequestIDs, the AMO architecture strictly guarantees that no banking transaction executes more than once.

Ultimately, this project highlights that while UDP provides a fast, lightweight transport layer, ensuring true transactional safety demands rigorous, application-level fault tolerance and state caching protocols.

Appendix:

The following appendix contains direct, unedited console screenshots validating the empirical findings discussed in Section 5. All experiments were conducted with the reply drop probability being set to 50%.

ALO

1. Account Creation

```
--- Distributed Bank Menu ---
1. Open Account
2. Close Account
3. Deposit
4. Withdraw
5. Monitor
6. Check Balance
7. Transfer Funds
q. Quit
Choice: 1
Name: Marcus
Password: ntu
Currency (USD, JPY, or SGD): USD
Initial Balance: 1000
[TIMEOUT] No response. Retrying (1/5)...
[TIMEOUT] No response. Retrying (2/5)...

[SERVER RESPONSE]: ACCOUNT CREATION SUCCESSFUL, account number: 2
```

The server successfully processes the original request, spawning a new user profile natively mapped to Account ID '0'. However, due to severe network congestion, the server's acknowledgment is dropped. The client subsequently times out and blindly retransmits the account creation packet twice. Operating identically and statelessly, the ALO server interprets both network-driven retries as entirely distinct registration attempts. Consequently, the server blindly creates two redundant 'ghost' accounts (Account ID '1' and Account ID '2'). The client finally receives the delayed reply for Account ID '2', leaving the backend database dangerously bloated with orphaned duplicates.

2. Withdrawal

```
--- Distributed Bank Menu ---
1. Open Account
2. Close Account
3. Deposit
4. Withdraw
5. Monitor
6. Check Balance
q. Quit
Choice: 4
Account Number: 2
Name: Marcus
Password: ntu
Currency (USD, JPY, or SGD): USD
Amount: 10
[TIMEOUT] No response. Retrying (1/5)...

[SERVER RESPONSE]: WITHDRAW SUCCESSFUL: Withdrawn 10.00 USD into account 2, new balance: 980.00 USD
```

The client requests to properly withdraw 10 USD from their account. The server correctly processes the math, deducting the requested funds, but the outbound confirmation reply is forcefully dropped by the network. Assuming a failure, the client times out and automatically vectors a retransmission. The stateless ALO server receives the duplicated packet and instantly interprets it as a completely new, discrete transaction intent. It processes the withdrawal a second time, illegally deducting another 10 USD from the user's ledger. The client finally receives the delayed SUCCESS packet, entirely unaware that the network timeout implicitly forced a double deduction against their account resulting in a final balance of 980 USD.

3. Close Account

```
--- Distributed Bank Menu ---
1. Open Account
2. Close Account
3. Deposit
4. Withdraw
5. Monitor
6. Check Balance
7. Transfer Funds
q. Quit
Choice: 2
Name: Marcus
Account Number: 0
Password: ntu
[TIMEOUT] No response. Retrying (1/5)...

[SERVER RESPONSE]: ACCOUNT CLOSURE ERROR: Error: Invalid credentials or account not found.
```

Client successfully issues an “Account Close” request, which the server processes,effectively closing the account. However, because the server’s acknowledgment reply is lost in the network, the client subsequently times out and automatically retransmits the closure request. The server attempts to close the account a second time. Finding the account already deleted, it strictly returns an ERROR_ACCOUNT_NOT_FOUND message. This results in the client being falsely alerted that the transaction failed.

4. Deposit Amount

```
Initial Balance: 1000

[SERVER RESPONSE]: ACCOUNT CREATION SUCCESSFUL, account number: 0

--- Distributed Bank Menu ---
2. Close Account
3. Deposit
4. Withdraw
5. Monitor
6. Check Balance
7. Transfer Funds
q. Quit
Choice: 3
Account Number: 0
Name: Marcus
Password: ntu
Currency (USD, JPY, or SGD): USD
Amount: 10
[TIMEOUT] No response. Retrying (1/5)...

[SERVER RESPONSE]: DEPOSIT SUCCESSFUL: Deposited 10.00 USD into account 0, new balance: 1020.00 USD
```

The client submits a request to confidently deposit 10 USD into an account containing 1000 USD. The server successfully intercepts the packet and increments the internal balance to perfectly match the 1010 USD trajectory. However, the subsequent network acknowledgment is forcefully dropped. Presuming a transmission failure, the client times out and aggressively re-transmits the identical deposit command. Operating strictly without duplicate resolution, the ALO server immediately commits the secondary packet as a completely distinct, discrete transaction. A secondary 10 USD is autonomously credited into the database. When the final success marker reaches the client, the user is presented with a falsely verified, artificially corrupted final balance of 1020 USD.

5. Monitor Account

```
Choice: 5
Monitor Interval (Example usage - 1hrs 30mins 5secs): 40s
[TIMEOUT] No response. Retrying (1/5)...

[SERVER RESPONSE]: ACK: Server will send updates for 40 seconds

[MONITOR UPDATE]: ACCOUNT CREATION SUCCESSFUL, account number: 0

[MONITOR UPDATE]: ACCOUNT CREATION SUCCESSFUL, account number: 0

[MONITOR UPDATE]: ACCOUNT CREATION SUCCESSFUL, account number: 1

[MONITOR UPDATE]: ACCOUNT CREATION SUCCESSFUL, account number: 1

[MONITOR UPDATE]: ACCOUNT CREATION SUCCESSFUL, account number: 2

[MONITOR UPDATE]: ACCOUNT CREATION SUCCESSFUL, account number: 2
```

```
--- Distributed Bank Menu ---
1. Open Account
2. Close Account
3. Deposit
4. Withdraw
5. Monitor
6. Check Balance
7. Transfer Funds
q. Quit
Choice: 1
Name: mar
Password: mar
Currency (USD, JPY, or SGD): SGD
Initial Balance: 10
[TIMEOUT] No response. Retrying (1/5)...
[TIMEOUT] No response. Retrying (2/5)...

[SERVER RESPONSE]: ACCOUNT CREATION SUCCESSFUL, account number: 2
```

LEFT: Monitoring Client, RIGHT: Client using services

Monitor service took 1 retry (2 requests), and thus for any operation, the monitoring client will receive one duplicate update (total of 2). The open account operation took 2 retries (3 requests). Therefore, as expected, the monitoring client receives 6 total updates, with 3 of them being duplicates.

6. Check Balance

```
--- Distributed Bank Menu ---
1. Open Account
2. Close Account
3. Deposit
4. Withdraw
5. Monitor
6. Check Balance
7. Transfer Funds
q. Quit
Choice: 6
Account Number: 0
Password: mar
[TIMEOUT] No response. Retrying (1/5)...
[TIMEOUT] No response. Retrying (2/5)...
[TIMEOUT] No response. Retrying (3/5)...

[SERVER RESPONSE]: CHECK BALANCE SUCCESSFUL: Account 0 has a balance of 10.00
```

```
Checked balance for account 0

size: 23
00 00 00 06 00 00 00 07 53 55 43 43 45 53 53 00 00 00 00 41 20 00 00
[SIMULATION] Dropped outgoing reply to client.
Checked balance for account 0

size: 23
00 00 00 06 00 00 00 07 53 55 43 43 45 53 53 00 00 00 00 41 20 00 00
[SIMULATION] Dropped outgoing reply to client.
Checked balance for account 0

size: 23
00 00 00 06 00 00 00 07 53 55 43 43 45 53 53 00 00 00 00 41 20 00 00
[SIMULATION] Dropped outgoing reply to client.
Checked balance for account 0

size: 23
00 00 00 06 00 00 00 07 53 55 43 43 45 53 53 00 00 00 00 41 20 00 00
```

LEFT: Client, RIGHT: Server

Client ends up sending 3 retransmissions and succeeds on the 4th request sent. The server checks the balance for the provided account number. Since this operation is idempotent (simply reading the balance off from the account), the server correctly returns the same balance (indicated by the same response message byte array displayed) to the client.

7. Transfer Funds

```
--- Distributed Bank Menu ---
1. Open Account
2. Close Account
3. Deposit
4. Withdraw
5. Monitor
6. Check Balance
7. Transfer Funds
q. Quit
Choice: 7
Source Account Number: 2
Password: mar
Destination Account Number: 4
Amount to Transfer: 3

[SERVER RESPONSE]: TRANSFER SUCCESSFUL: Sent 3.00 USD from Account 2 (new balance: 7.00 USD) -> Credited 448.50 JPY to Account 4 (new balance
: 458.50 JPY)
```

```
--- Distributed Bank Menu ---
1. Open Account
2. Close Account
3. Deposit
4. Withdraw
5. Monitor
6. Check Balance
7. Transfer Funds
q. Quit
Choice: 7
Source Account Number: 4
Password: jh
Destination Account Number: 2
Amount to Transfer: 3
[TIMEOUT] No response. Retrying (1/5)...

[SERVER RESPONSE]: TRANSFER SUCCESSFUL: Sent 3.00 JPY from Account 4 (new balance: 452.50 JPY) -> Credited 0.02 USD to Account 2 (new balance
: 7.04 USD)
```

TOP: Transfer from Account 2 to 4 of 3 USD. BOTTOM: Transfer from Account 4 to 2 of 3 JPY.

Two accounts were created - account 2 with initial balance of 10 USD, and account 4 with initial balance of 10 JPY. An initial successful transfer involved sending 3 USD from account 2 to account 4, resulting in account 2's balance being reduced to 7 USD, and account 4's balance increasing from 10 JPY to 458.50 JPY (where 3 USD = 448.50 JPY). A subsequent transfer

involves transferring 3 JPY back to account 2. This operation is executed twice due to the server's reply being dropped, and as expected account 4's balance decreases by 6 JPY to 452.50 JPY, and account 2's balance increases by 6 JPY to 7.04 USD.

AMO

1. Account Creation

```
--- Distributed Bank Menu ---
1. Open Account
2. Close Account
3. Deposit
4. Withdraw
5. Monitor
6. Check Balance
7. Transfer Funds
q. Quit
Choice: 1
Name: Marcus
Password: ntu
Currency (USD, JPY, or SGD): USD
Initial Balance: 1000
[TIMEOUT] No response. Retrying (1/5)...

[SERVER RESPONSE]: ACCOUNT CREATION SUCCESSFUL, account number: 0
```

The client initiates an 'Open Account' request, which the server processes, allocating a new memory structure and generating Account ID '0'. However, the server's initial success reply drops, triggering a timeout and retransmission from the client. Under ALO, the server would blindly interpret this retry as a second user, erroneously generating Account ID '1'. Instead, the AMO server successfully intercepts the duplicate RequestID, halting the redundant allocation logic, and safely returns the locally cached original reply. The client successfully receives the original Account ID '0' on the first retry, ensuring data consistency and preventing ghost accounts from flooding the backend.

2. Withdrawal

```
--- Distributed Bank Menu ---
1. Open Account
2. Close Account
3. Deposit
4. Withdraw
5. Monitor
6. Check Balance
7. Transfer Funds
q. Quit
Choice: 4
Account Number: 0
Name: Marcus
Password: ntu
Currency (USD, JPY, or SGD): USD
Amount: 10
[TIMEOUT] No response. Retrying (1/5)...
[TIMEOUT] No response. Retrying (2/5)...

[SERVER RESPONSE]: WITHDRAW SUCCESSFUL: Withdrawn 10.00 USD into account 0, new balance: 990.00 USD
```

The client requests a $10.00 withdrawal from an account holding $1000.00. The server successfully deducts the funds, locking the balance at $990.00, but sustains two consecutive reply drops over the highly congested network. Consequently, the client times out and retransmits the withdrawal command twice. Relying on its state history, the AMO server identifies both retransmissions as duplicates mapped to the initial RequestID. It rejects both execution attempts, actively shielding the user from an erroneous triple-deduction that would have plunged the balance to $970.00. The server sequentially re-transmits the cached reply until the network stabilizes on the second retry, delivering a completely accurate final balance of precisely $990.00.

3. Close Account

```
--- Distributed Bank Menu ---
1. Open Account
2. Close Account
3. Deposit
4. Withdraw
5. Monitor
6. Check Balance
7. Transfer Funds
q. Quit
Choice: 2
Name: Marcus
Account Number: 0
Password: ntu
[TIMEOUT] No response. Retrying (1/5)...
[TIMEOUT] No response. Retrying (2/5)...

[SERVER RESPONSE]: ACCOUNT CLOSURE SUCCESSFUL: Account 0 closed successfully.
```

The client successfully issues an 'Account Close' request which the server properly deletes. When the server’s initial 'SUCCESS' acknowledgment drops in transit, the client times out and retransmits the request. Upon receiving the duplicate request (Retry 1), the stateful AMO server detects the duplicate RequestID natively. Instead of incorrectly attempting to delete an

already-deleted account, the server bypasses the application logic and simply re-transmits its locally cached 'SUCCESS' reply. Because of the simulated heavy packet loss, this cached reply is also dropped, prompting a second timeout and retransmission (Retry 2). The server once again seamlessly filters the duplicate and transmits the cached reply. This reply eventually succeeds, ensuring that the client receives the accurate, original success message without any false positive errors.

4. Deposit Amount

```
--- Distributed Bank Menu ---
1. Open Account
2. Close Account
3. Deposit
4. Withdraw
5. Monitor
6. Check Balance
7. Transfer Funds
q. Quit
Choice: 3
Account Number: 1
Name: Marcus
Password: ntu
Currency (USD, JPY, or SGD): USD
Amount: 10
[TIMEOUT] No response. Retrying (1/5)...

[SERVER RESPONSE]: DEPOSIT SUCCESSFUL: Deposited 10.00 USD into account 1, new balance: 1010.00 USD
```

After attempting to deposit 10 USD into the account the server successfully processes the transaction, advancing the internal balance to 1010 USD. but the outgoing acknowledgement is forcefully dropped by the unstable network. Unaware of the initial success, the client times out and retransmits the deposit command. Rather than blindly re-executing this operation which would have incorrectly duplicated the deposit and inflated the balance to $1020.00 under ALO, the stateful AMO architecture intercepts the duplicate RequestID. It rejects the redundant execution and cleanly returns the previously cached success reply. The client parses this recovered message, confirming a strictly accurate, single-execution final balance of $1010.00.

5. Monitor Account

```
--- Distributed Bank Menu ---
1. Open Account
2. Close Account
3. Deposit
4. Withdraw
5. Monitor
6. Check Balance
7. Transfer Funds
q. Quit
Choice: 5
Monitor Interval (Example usage - 1hrs 30mins 5secs): 40s
[TIMEOUT] No response. Retrying (1/5)...

[SERVER RESPONSE]: ACK: Server will send updates for 40 seconds

[MONITOR UPDATE]: ACCOUNT CREATION SUCCESSFUL, account number: 1
```

When another user opens a separate account, the monitoring client will only receive one update as expected, despite in this case, there being one retry (2 transmitted requests).

6. Check Balance

```
--- Distributed Bank Menu ---
1. Open Account
2. Close Account
3. Deposit
4. Withdraw
5. Monitor
6. Check Balance
7. Transfer Funds
q. Quit
Choice: 6
Account Number: 0
Password: mar
[TIMEOUT] No response. Retrying (1/5)...

[SERVER RESPONSE]: CHECK BALANCE SUCCESSFUL: Account 0 has a balance of 15.00
```

```
Checked balance for account 0

size: 23
00 00 00 06 00 00 00 07 53 55 43 43 45 53 53 00 00 00 00 41 70 00 00
[SIMULATION] Dropped outgoing reply to client.
Duplicate request 127.0.0.1:54473-5 detected. Re-sending cached reply.

size: 23
00 00 00 06 00 00 00 07 53 55 43 43 45 53 53 00 00 00 00 41 70 00 00
```

LEFT: Client, RIGHT: Server

On receiving the same retransmitted request, the server will send the cached response instead as shown in the screenshot on the right.

7. Transfer Funds

```
--- Distributed Bank Menu ---
1. Open Account
2. Close Account
3. Deposit
4. Withdraw
5. Monitor
6. Check Balance
7. Transfer Funds
q. Quit
Choice: 7
Source Account Number: 0
Password: mar
Destination Account Number: 2
Amount to Transfer: 1
[TIMEOUT] No response. Retrying (1/5)...
[TIMEOUT] No response. Retrying (2/5)...
[TIMEOUT] No response. Retrying (3/5)...

[SERVER RESPONSE]: TRANSFER SUCCESSFUL: Sent 1.00 USD from Account 0 (new balance: 9.00 USD) -> Credited 149.50 JPY to Account 2 (new balance
: 159.50 JPY)
```

Two accounts were created: Account 0 with an initial balance of 10 USD, Account 2 with an initial balance of 10 JPY. Despite retransmission, the transfer operation only executes once, wherein only 1 USD is transferred as intended. New balances correctly reflect this.